Tower — From Implementation Research to Implementation Fidelity & Decision Fit

Date: October 19, 2026
Scope: The discussion following Tower's workflow-graph prior-art research, covering approximately the final 1–2 hours of conversation.
Status: Analytical record and architectural discovery. Not a new product specification or implementation authorization.

---

1. Where the discussion began: the workflow-graph research

1.1 The research artifact

The latest research pass had been completed in Tower's repository:

- PR: "#131 — Research the workflow graph editor and walker" (https://github.com/bywale-com/omcoda_tower/pull/131)
- Branch: "cursor/tower-workflow-graph-prior-art-0b4d"
- Document: "docs/TOWER-WORKFLOW-GRAPH-PRIOR-ART.md"

This investigation followed two earlier documents:

- "TOWER-LOAD-BEARING-ARCHITECTURE-DOSSIER.md"
- "TOWER-ARCHITECTURE-PRIOR-ART.md"

Our research initiative had originally been intended to counter unnecessary reinvention.

The method was to identify Tower's load-bearing systems, study how established production software implements the same mechanisms, investigate those implementations' failure modes, and determine which proven solutions Tower should keep, adapt, or reuse.

Our optimization criteria remained:

1. Compactness.

2. Cost-effectiveness.
3. Minimal complexity.

The graph investigation was the first focused deep dive following the broader architecture research.

1.2 What the graph research established

The agent examined Tower's existing React Flow editor and custom workflow interpreter alongside actual source implementations from xyflow and n8n.

Its principal recommendation was to retain:

- "@xyflow/react" for the visual editor.
- "walkValidatedWorkflow()" for execution.
- The existing domain-specific node vocabulary.
- The distinction between Save, Run, and Launch.
- The existing representation of workflows in Postgres.

It recommended adapting several supporting mechanisms instead of adopting another execution framework.

The research uncovered specific implementation behaviours:

Graph validity

"isValidatedPipeline()" checked whether certain node kinds existed, not whether the graph's edges formed an executable topology.

A graph could contain invalid or ambiguous connections while passing the existing validation check.

Multiple incoming edges

The walker tracked visited nodes. When a node had multiple incoming edges, the first visit prevailed, and later arrivals were ignored.

This avoided infinite repetition but did not implement deliberate join semantics.

Cycles

The visited-node set prevented cycles from executing indefinitely, but invalid cycles were not necessarily rejected at authoring or activation.

Deletion

Different UI deletion paths could produce different resulting edge structures.

Wait and resume

The walker loaded "workflow.graph" when executing, including after resuming from a Wait.

Therefore, a workflow could begin under one graph definition and continue under another if the Operator edited the graph while the run was parked.

Run to here

Running to a selected node did not reconstruct execution from previously captured upstream results. It started at the Trigger and executed through the selected node.

This could repeat operations with actual business effects.

1.3 The initial architectural response

We initially concluded that the graph research was significantly better than the earlier broad pass because it had located genuine discrepancies between visual authoring, persisted graph structures, and execution semantics.

We identified graph revisioning as particularly consequential.

Consider:

- Monday: an automation begins using definition A.
- Tuesday: it reaches Wait.
- Wednesday: the Operator saves definition B.
- Thursday: the waiting run resumes and reads B.

The execution retains its identity but may no longer retain the instructions under which it began.

This suggested distinguishing three identities:

Identity| Meaning
Editable definition| What the Operator is currently modifying
Published configuration| What is authorized to begin new work
Execution configuration| What a particular running instance uses

A related finding made this more serious.

Saving an existing active workflow updates its stored graph while leaving its active status unchanged. Therefore, an edit can affect subsequent event-triggered executions without another explicit Launch.

We proposed that graph snapshots, revision references, or another mechanism might be necessary.

We also recognized that graph snapshots would not automatically preserve separately referenced configuration, such as rules, constants, templates, and firm overrides.

At this stage, we were interpreting discovered discrepancies mainly as engineering and architecture problems.

That framing was about to change.

---

1. The pivotal challenge: wasn't this already decided?

The next question changed the direction of the conversation.

The user challenged the assumption that the newly discovered edge cases required fresh product decisions.

The challenge was essentially:

«In what ways is the deeper methodology around Automations and Agents, already captured across the Markdown documents and historical logs, insufficient to resolve these questions?»

The underlying position was stronger than a request to search the documentation.

The deeper product idea should already constrain how implementation problems are resolved.

Rather than introducing a new product decision whenever engineering exposes ambiguity, we should first determine whether the original methodology already decides the issue—or makes the answer derivable.

This distinction became the central discovery of the next phase.

2.1 We returned to the original product record

We investigated several foundational sources, including:

- "docs/register/PHASE-TWO-LOG.md"
- "docs/register/AGENTS-FACE.md"
- "docs/register/AUTOMATIONS-FACE.md"
- "docs/register/PRIMITIVES.md"
- "docs/register/SYSTEMS-AUTOMATIONS-NODE-OUTPUT.md"
- "docs/register/SYSTEMS-ANALYST-LOG.md"
- "docs/register/SYSTEMS-CHANGE-PACKET-022.md"
- "docs/register/SYSTEMS-CHANGE-PACKET-023.md"

The critical reference was the original Agents flagship recorded on August 15, 2026, at 15:43 UTC, together with subsequent refinements and corrections.

These sources revealed that the product's conceptual architecture was far more developed than the recent prior-art research had reconstructed.

2.2 The foundational distinction between Agents and Automations

The original methodology established two different responsibilities.

Automations

Automations owns the orchestration of application operations:

- What event occurred?
- What operation should be performed?
- Which real application module performs it?
- What did that operation produce?
- What should happen based on its results?
- Should another operation or Agent be initiated?

An automation is not merely an outreach trigger.

It can coordinate real operations across the application.

Agents

Agents owns engagement toward an objective:

- Campaign authorship.
- Campaign objectives.
- Sequencing.
- Channels and content.
- Attempt cycles.
- Escalation rules.
- Completion and failure.
- Defaults and overrides.

An Agent is a sequence containing campaigns, not an isolated campaign.

Automations can assign an existing sequence or instantiate one from a template, but it does not acquire authority to redefine that sequence's internal structure.

The runtime

The underlying machine persists the relevant records, performs configured operations, enforces gates, and records consequences.

The result is an architecture with defined ownership boundaries.

The modules are not simply different interface panels. They have different kinds of authority.

This finding immediately constrained several recommendations from the graph research.

---

1. First worked example: outputs after a node and after Wait

The graph research had raised the question of what happens when a workflow resumes and its previous "chainWrites" map is no longer available.

Originally, we treated the permitted scope of earlier outputs as a potentially unresolved product decision.

The deeper methodology showed otherwise.

3.1 The original node-output law

"AUTOMATIONS-FACE.md", law 33, establishes that when a node completes, downstream execution can access the writes produced by that particular

operation.

"PRIMITIVES.md" describes the two layers:

The writes

The actual records belong to the real application module that produced them.

For example, an Audit operation produces the same records whether it was initiated through the Audit desk or an Automation Action.

The handle

The workflow retains a reference to the records produced by that particular execution.

The next node must not guess which audit it should read by searching for the latest audit associated with a firm.

The identity of the producing operation matters.

The product is not fundamentally about whether outputs travel through an n8n-style pipe or through database references.

Both can satisfy the same requirement.

The requirement is that the next node has access to what the actual preceding module execution produced.

3.2 What that means for Wait and resume

Consider:

"Trigger → Audit → Wait → If → Action"

The Audit produces a set of records.

The workflow parks.

When it resumes, the If still needs the appropriate Audit output.

The methodology implies that pausing execution must not destroy a valid authored reference to an earlier node's results.

Therefore, the question is no longer:

Should Tower preserve those outputs after Wait?

The more appropriate question is:

Which established implementation mechanism preserves those outputs with the least complexity and sufficient recovery guarantees?

Possible technical implementations include durable write-set references, persisted execution context, or reconstruction from records already associated with the run.

Those are engineering alternatives.

The product guarantee already constrains them.

3.3 The general lesson

This was our first concrete demonstration that we had been mistaking implementation freedom for product uncertainty.

The methodology fixes the required behaviour while leaving flexibility about the internal mechanism.

The research should explore that flexibility, not reopen the requirement.

---

1. Second example: graph branching and the difference between technical convenience and product semantics

The graph research recommended rejecting multi-parent nodes and maintaining a constrained workflow topology.

Technically, that recommendation was defensible.

However, the methodology introduced an important qualification.

4.1 Rule output is not necessarily exclusive

"AUTOMATIONS-FACE.md", particularly laws 21 and 22, describes rule outputs as a lattice of potentially coexisting result classes.

For example, an evaluation may produce both:

- A re-engagement signal.
- A reactivation signal.

The presence of one does not necessarily erase the other.

Different conditions may subsequently produce different actions based on those classes.

This means the product's general rule-output semantics cannot simply be reduced to a single mutually exclusive selection.

4.2 We distinguished several separate mechanisms

An If is a binary condition.

A Switch chooses among configured paths according to its semantics.

A rule-output lattice permits several independently meaningful results.

Fan-out permits one execution result to initiate multiple downstream operations.

A join combines or coordinates multiple incoming execution paths.

These are not equivalent.

Supporting fan-out does not automatically require supporting joins.

Likewise, the fact that the current Switch chooses one route does not imply that every workflow operation must have single-path semantics.

4.3 The correction to the research method

The permitted graph topology should be derived from Tower's product semantics.

It should not be derived solely from the subset of topologies the current walker happens to implement conveniently.

The correct sequence is:

1. Establish which workflow behaviours the original methodology promises.
2. Identify the graph structures required to express them.
3. Reject additional structures only when they are unnecessary to those requirements.
4. Adopt proven graph-validation mechanisms that enforce the resulting contract.

A join-free topology may still be the correct choice.

But it must be justified by the product model, not merely by a desire to avoid implementation work.

This distinction remains relevant to evaluating PR #131's recommendation.

---

1. Third example: idempotency versus deliberate repetition

The graph research also raised the possibility that repeated manual Runs could create duplicate enrollments or other effects.

Initially, we interpreted this as an idempotency question.

The methodology forced a more precise distinction.

5.1 Two apparently identical executions may represent different intentions

Case A: Retry

A user initiates one operation.

The worker crashes, the network times out, or a request is delivered again.

The infrastructure repeats the operation.

This should ordinarily preserve the identity of the original logical request rather than unintentionally repeat its effects.

Case B: A new deliberate operation

An Operator intentionally initiates another run or another engagement.

The new operation may use the same contact, sequence, or workflow.

But it may nevertheless represent a separate authorized business action.

5.2 The resulting principle

Deduplicate repeated attempts at the same intended operation, not every operation that happens to resemble an earlier one.

For Agents, that may require distinguishing:

- Contact.
- Enrollment.
- Campaign.
- Objective.
- Attempt.
- Message.
- Workflow execution.

A permanent uniqueness constraint on a broad combination such as firm, contact, and sequence could incorrectly prevent a legitimate new pursuit.

The business model determines which actions may repeat.

The implementation determines how unintended duplication is prevented.

This was another case where product methodology narrowed the engineering question substantially.

---

1. Fourth example: the methodology had already defined failure

This became the strongest historical demonstration.

We inspected two Systems Analyst change packets.

6.1 Packet 022: objective completion is a Local event

"SYSTEMS-CHANGE-PACKET-022.md" describes how a completed objective should be tied to the specific campaign and enrollment pursuing it.

The original problem was that some completion checks were too broad.

For example, a previously booked meeting might be mistaken for fulfillment of an entirely different current objective.

The packet established that completion must carry or resolve the identity of the objective it satisfies.

Its mechanism was to use the existing event and database architecture:

- Stamp the pursuit's objective identity onto relevant outbound or completion records.
- Match the eventual completion against the correct objective.
- Treat an unstamped, unrelated completion as insufficient.
- Record the completion.
- Advance the campaign cursor.
- Expose objective completion as a subscribable event.

This avoided inventing an entirely separate goal-completion architecture.

The product principle—an objective is an identifiable completion event—determined the mechanism's required behaviour.

6.2 Packet 023: failure means stop spending, not stop listening

The original founder decision was especially significant.

An exhausted campaign has consumed the resources allocated to pursuing an objective.

But that does not necessarily mean the objective can no longer be achieved.

Therefore:

State| Outbound pursuit| Inbound completion
Pursuing| Continues| Accepted
Exhausted| Stops| Still accepted
Nullified, in the proposed future escrow model| Stops| Refused

The reasoning came from Tower's intended commercial model.

Objectives are potential billable outcomes.

If the Agent exhausts its attempts and stops contacting someone, but that person later independently completes the objective, the completion can still matter.

Therefore, exhaustion should not automatically:

- Remove the current campaign cursor.
- Invalidate links required to complete the objective.
- Prevent the system from recognizing a later completion.
- Automatically begin a replacement campaign.

The implementation packets converted those principles into concrete state transitions and database writes.

6.3 Why this example mattered

A generic workflow engine might treat failure as the end of execution.

Tower's business model gives failure a different meaning.

Consequently, we should research mature workflow systems for their durable execution, scheduling, concurrency, and recovery mechanisms.

But we should not import their business semantics without checking whether those semantics belong in Tower.

Established engineering mechanisms are reusable. Established product meanings are not automatically transferable.

This made the role of external research much clearer.

---

1. Fifth example: agency-wide configuration and inheritance

We then returned to the broader configuration architecture that underlies both Agents and Automations.

The Agents methodology already specifies a cascading hierarchy of defaults and overrides.

The relevant conceptual hierarchy is:

"Global → Sequence → Campaign → Channel"

Rules are defined at the level they govern.

For example:

- Global rules supply defaults.
- A sequence can override applicable global settings.
- A campaign can override applicable inherited settings.
- Channel rules apply to that medium.
- Attempt rules are attached to their appropriate cycle or governing scope, rather than indiscriminately placed on channel nodes.

The methodology describes fallback in CSS-like terms.

This is an underlying configuration-resolution model, not merely a preferred UI arrangement.

7.1 What it already answers

It determines where configuration belongs.

It determines which module owns which kinds of configuration.

It determines that clients and sequences should not need independently duplicated copies of every default.

It also constrains how an Automation's decision to assign an Agent interacts with that Agent's internal rules.

7.2 What remained distinct

Configuration inheritance answers:

Which applicable value wins?

It does not necessarily answer:

When should that value be resolved?

Consider a campaign that is already running when an agency changes a global rule.

Some authored behaviour may need to remain stable for the execution's lifetime.

Other information, such as an opt-out or halt, must be enforced according to the latest applicable policy.

Reference data may have still another temporal rule.

Therefore, we recognized that configuration resolution has at least two dimensions:

1. Scope: Which default or override applies?
2. Time: Which version or current state applies to this operation?

This is a more precise research problem than asking, without context, whether Tower needs configuration versioning.

The methodology substantially constrains the answer, while some category-specific temporal decisions may still require explicit formulation.

---

1. Sixth example: historical decisions can supersede other historical decisions

Another important discovery came from the Systems Analyst log.

8.1 The August 21 correction

At approximately 10:10 UTC on August 21, a proposed solution would have required the objective selected by an Automation Action to agree with the objective authored inside the Agent sequence.

It seemed like a reasonable consistency check.

The proposed implementation would block configurations that appeared incompatible.

But at 10:35 UTC, the founder rejected that coupling.

An Agent sequence is capable of containing multiple campaigns and objectives.

The Automation's entry binding does not govern the sequence's internal authorship.

The Agent remains agnostic about the Automation that invokes it.

The proposed compatibility gate was retracted.

8.2 The significance

This demonstrated a proposed decision that seemed locally sound but did not fit the established system.

The error was not a lack of engineering knowledge.

It was failure to preserve an existing authority boundary.

More importantly, some earlier documentation still contained language compatible with the rejected idea.

A researcher reading only an older document could reconstruct the wrong product rule.

Therefore, storing decisions chronologically is insufficient.

The architecture must recognize relationships between decisions, including:

- Which decision is authoritative.
- Which later decision supersedes an earlier one.
- Which module or behaviour the decision governs.
- Which other decisions constrain it.
- Whether a recommendation contradicts an established invariant.

This became an early sign of what we would later call Decision Fit.

---

1. Our first methodological conclusion

After examining these cases, we revised the research process.

The previous process had implicitly been:

"Existing code → External research → Engineering recommendation"

That was inadequate.

It allowed the current implementation to become the starting assumption about what the product should do.

It also allowed external implementations to determine possible product behaviour before Tower's original decisions had been reconstructed.

We proposed a different sequence:

"Product methodology → Existing implementation → External prior art → Compliant implementation"

The responsibilities became clearer.

Product methodology

Establishes what must be true and why.

Existing implementation

Shows how Tower currently attempts to make those requirements true.

External prior art

Shows how experienced engineers and mature systems solve comparable technical problems.

Engineering recommendation

Selects the smallest sound implementation that preserves the established requirements.

9.1 Four classes of apparent uncertainty

We proposed distinguishing four cases.

Category| Meaning| Appropriate response
Already decided| An authoritative decision explicitly answers the issue| Apply it
Derivable| The answer follows from existing product principles| Demonstrate the derivation
Implementation freedom| Several mechanisms could satisfy the same requirement| Research and choose
Genuine product silence| Multiple materially different behaviours fit the existing model| Seek a new product decision

This was a significant correction.

Not every unanswered question in an engineering document is an unanswered product question.

The relevant answer may already exist elsewhere in the decision history.

And even where no sentence states the answer explicitly, established principles may determine it.

---

1. The apparent remaining question: what is Run?

We had previously raised one issue that seemed to remain unresolved.

Does the Run control in Automations represent:

- A live execution?
- A simulated test?
- A controlled test using specimen inputs?
- Some explicit combination of these?

The user responded that this had almost certainly already been addressed during the build phase.

There was a specific recollection of a Test function and versioned test templates, possibly associated with individual nodes.

Rather than treating the memory as a new requirement, we returned to the historical implementation record.

This produced another major discovery.

---

1. Rediscovery of the original Test system

11.1 PR #99: versioned event test contracts

We found "PR #99 — Fill Automations Test from the event contract" (https://github.com/bywale-com/omcoda_tower/pull/99).

This implemented a versioned test-data system.

The underlying migration was:

"supabase/migrations/017_event_test_contract.sql"

It introduced:

"event_write_set"

Describes the table-level write-set associated with a named event and contract version.

"event_test_data"

Contains specimen records associated with that event contract version.

The central identity is based on event name, version, and the relevant table or specimen record.

A shipping version was not supposed to be silently overwritten; revised contracts required a new version.

The implementation could retrieve the latest version of an event's contract.

11.2 Two sources of test input

The original methodology distinguished two doors.

Door 1: a real event instance already exists

Suppose a contact import has already happened.

The Test function can use records produced by that event.

The function must not create a new import merely to obtain data.

Door 2: the event has not happened yet

The system uses specimen values that conform to the real event's write-set.

For example, a contact import specimen contains the same kinds of records a real import would produce.

Those records are not inserted into the live application merely by requesting test data.

In both cases, the purpose is to populate the Trigger's output with meaningful fields and values so the Operator can configure downstream mappings.

The specimen must reflect the actual event shape, not a separately invented demonstration schema.

11.3 What the first implementation originally did

PR #99 initially placed Test on the canvas.

It could project the downstream chain without committing the resulting operations.

This was a genuine simulation-oriented interpretation.

But that was not the final product decision.

The original Test model was subsequently revised.

---

1. The decisive historical correction: PR #105

We located "PR #105 — Move Test into the trigger; split Run from Launch" (https://github.com/bywale-com/omcoda_tower/pull/105).

More importantly, we recovered the original Product Manager packets in "PR #96" (https://github.com/bywale-com/omcoda_tower/pull/96):

- "docs/register/TEST-PACKET.md"
- "docs/register/RUN-PACKET.md"

These preserve the founder's decisions from August 19, 2026, at 12:04 UTC.

The correction arose from using the actual software.

The founder observed that the function labeled Test was not really a complete test of the workflow.

It was providing data to the Trigger.

Therefore, Test should live inside the Trigger node and be named according to its actual purpose.

A separate Run control should execute the graph.

Launch should arm the workflow.

12.1 The three settled operations

Operation| Intended behaviour
Provide test data| Populate the Trigger with an existing event instance or versioned specimen; no execution or business writes
Run| Execute the workflow now, using the provided Trigger data, through the real interpreter
Launch| Arm the workflow to execute when future matching events occur

This was not a hypothetical design.

It was an explicit redesign that reached application code.

12.2 Run has two grains

The original specification also distinguished:

Canvas Run

Executes the chain from its beginning until completion or until something prevents further execution.

Run to here

Executes through a selected node.

The historical design referenced n8n as a useful interface precedent.

However, Tower's real interpreter remained authoritative for execution.

There was no requirement to introduce a second execution engine.

12.3 The original sandbox understanding

The recovered RUN packet contains another important decision.

The founder understood the application environment as a sandbox in which manually running configured workflows was acceptable.

Consequently, Run was intended to perform real application operations and produce real writes within that environment.

It was not necessarily a no-effect preview.

This changed our interpretation considerably.

Run was designed as immediate execution in an intended sandbox context, not as a synonym for Test.

---

1. The distinction between intended behaviour and current implementation

We then verified the relevant server and UI paths.

13.1 Current Trigger test-data behaviour

The present "AntAutomationWorkbench.tsx" exposes "handleProvideTriggerTestData()".

It obtains data through the event-test contract API.

The server implementation lives in:

"server/wire-service/eventTest.ts"

The implementation:

- Finds the latest contract version.
- Attempts to retrieve existing event records.
- Falls back to versioned specimens when necessary.
- Returns the write-set and data.
- Does not insert the provided specimens as business records.

This is substantially consistent with the original Test architecture.

However, the original design wanted the ability to choose a specific existing event instance.

The current reader generally selects recent records automatically.

Thus, an identifiable implementation gap remains.

13.2 Current Run behaviour

The workbench's "handleRun()":

1. Reads the writes already supplied to the Trigger.
2. Refuses to run if the Trigger has no data.
3. Saves the graph.
4. Starts a manual workflow run.
5. Displays execution statuses on the canvas.

The backend creates a "workflow_run" and calls:

"walkValidatedWorkflow()"

This is real server execution.

For an Agent Action, the walker can call:

"enrollAndActuate()"

That function creates enrollment and active-attempt records, even when outbound delivery is configured as dry-run.

External sends are subject to separate sending gates.

The current sending infrastructure defaults to "dry_run", while actual live delivery requires explicit configuration and a contact allowlist.

Therefore, there are separate questions:

- Does Run execute the real interpreter? Yes.
- Does Provide test data write business records? It is designed not to.
- Can Run write business records? Yes.
- Does Run necessarily send externally? No; send policy governs that.
- Is Run inherently isolated from production data simply because it is called Run? No such guarantee was established.

13.3 What was actually versioned?

The user's memory of versioned test templates was substantively correct.

The versioned mechanism exists.

However, what we directly verified was versioning per event contract, not independently per graph-node instance.

We did not find a separate general-purpose versioned test-template system covering every individual Action, Get, Rule, and If node.

It remains possible that a broader idea appeared elsewhere in historical discussions.

But the confirmed implemented architecture is event-based.

This distinction matters because the relevant event contract can be shared across multiple Trigger nodes.

There is no need to duplicate the same event specification for every node instance.

13.4 The conclusion

The original product distinction between Test, Run, and Launch was not unresolved.

It had been explicitly discussed, revised, implemented, and recorded.

The remaining problems concerned whether the implementation fully fulfilled that decision.

That realization immediately changed the vocabulary of the entire conversation.

---

1. Eureka: Implementation Fidelity

The user identified the first central concept:

Implementation Fidelity.

This precisely names the problem we had been encountering.

A product decision is made.

Its rationale is established.

The decision is translated into specifications.

Engineering implements the feature.

But the resulting software does not fully conform to the original decision.

The discrepancy may be subtle.

A button exists, but the underlying operation differs.

A setting is editable, but runtime execution does not honor it.

A workflow runs, but its outputs are not preserved according to the authored contract.

A configuration inherits defaults in the UI, but the relevant server reader ignores the override.

A completion event exists, but its identity is insufficient to associate it with the correct objective.

These are not necessarily failures to specify the product.

They are failures to carry a settled decision into operational reality.

14.1 Formal definition

Implementation Fidelity is the degree to which an implementation preserves the requirements, invariants, authority boundaries, and intended behaviours established by its governing product decisions.

The essential comparison is:

"Established decision → Actual implementation"

The central question:

Does the system actually do what we already decided it should do?

14.2 Why this explained so many earlier problems

Tower had accumulated extensive methodology, logs, systems packets, UI specifications, and founder corrections.

Yet newly implemented functionality could still depart from those materials.

The existence of the correct decision somewhere in the repository did not guarantee faithful implementation.

The problem was therefore not simply missing content.

The decision did not exercise reliable structural authority over the implementation.

Documentation existed. Conformance was not reliably enforced.

14.3 The Test/Run/Launch example

The original requirements were precise.

Provide test data belonged inside the Trigger.

Run executed immediately.

Launch armed future event-driven execution.

The live implementation could now be compared with those decisions.

Any differences would be categorized as implementation gaps rather than opportunities to invent different product behaviour.

This is the practical meaning of Implementation Fidelity.

---

1. The second shade: Decision Fit

The user then named a second, closely related problem:

Decision Fit.

The two ideas explain different moments in the engineering process.

Implementation Fidelity examines what has already been built.

Decision Fit examines a proposed solution before it becomes part of the system.

15.1 The motivating pattern

Our prior-art research had uncovered discrepancies and failure modes.

We then raised follow-up questions.

Some were legitimate technical investigations.

But others implicitly asked the founder to make decisions that should already have followed from the existing architecture.

For example:

- Should outputs survive Wait?
- What should failure mean?
- Should Automations govern an Agent's internal objective?
- What is the intended meaning of Run?
- Should a particular new validation constraint be introduced?

Some of those questions had already been explicitly answered.

Others were derivable from established principles.

What was missing was a mechanism for determining whether a proposed solution fit the governing decisions.

15.2 Formal definition

Decision Fit is the degree to which a proposed decision or engineering solution is consistent with the existing system of authoritative decisions, constraints, and intended behaviours.

Its central comparison is:

"Proposed solution → Established decision system"

The central question:

Does this solution belong in the system we've already designed?

15.3 Decision Fit is not simply agreement with existing code

This distinction is crucial.

Suppose a new mechanism is compatible with the current walker but contradicts the original product methodology.

It may fit the code.

But it fails Decision Fit.

Similarly, an engineering proposal may successfully resolve a local bug while introducing a constraint that violates another module's ownership.

That is still a failure of fit.

Decision Fit is evaluated against the intended architecture, not merely the existing implementation.

15.4 The August 21 objective-binding correction as an example

The proposed compatibility gate between an Automation's chosen objective and an Agent sequence seemed reasonable.

It might even have reduced certain runtime inconsistencies.

But it contradicted the established agnosticism of Agents sequences.

The local proposal failed to fit the overall product model.

The later founder correction restored the correct authority boundary.

Decision Fit should ideally identify that contradiction before engineering implements it.

---

1. The relationship between the two problems

We arrived at a concise distinction.

| Implementation Fidelity| Decision Fit
Direction| Retrospective| Prospective
Starting point| Existing implementation| New problem or proposed solution
Reference| Governing decisions| Governing decisions
Question| Did we implement the decision correctly?| Does this proposed decision belong?
Failure| Software diverges from intent| New choices contradict or unnecessarily redefine intent
Desired outcome| Verified conformance| A sound, compatible engineering choice

They are two operations over the same underlying structure.

Implementation Fidelity follows decisions forward into code.

Decision Fit works from a problem or proposal back through the decisions that constrain it.

Both depend on being able to identify which decisions actually govern a particular part of the system.

---

1. Why more documentation is not the answer

The user then made the larger architectural observation:

Both problems can be solved with the right architecture, not simply more content or more context.

This was the conceptual climax of the discussion.

Tower's experience demonstrated that decisions could exist across extensive documentation and still fail to govern implementation or subsequent recommendations.

A sufficiently detailed document is not equivalent to an enforceable decision structure.

Likewise, giving an agent every historical conversation does not automatically ensure that it can distinguish:

- Current decisions from obsolete decisions.
- Product invariants from implementation details.
- Explicit requirements from speculative suggestions.
- Local decisions from constraints imposed by neighboring modules.
- Genuine silences from questions already answered elsewhere.

The needed improvement is not simply a larger context window.

It is a more reliable architecture of decision authority and traceability.

---

1. The emerging architecture: three distinct layers

We proposed distinguishing three layers.

18.1 Decision architecture

This represents what must be true.

It contains the authoritative product decisions and their relationships.

Its important dimensions include:

- Decision identity.
- Rationale.
- Scope.
- Module ownership.
- Required behaviour.
- Constraints and invariants.
- Relationships to other decisions.
- Supersession and correction.
- The degree of freedom left to engineering.

For example:

A completed Automation Action exposes the records produced by its particular execution.

That decision constrains the workflow walker, node-output references, persistence, and resumption behaviour.

It does not prescribe a particular database schema.

18.2 Implementation architecture

This represents how those requirements are currently made operational.

It includes:

- Source files.
- Functions.
- APIs.
- Tables.
- Workers.
- Dependencies.
- Execution paths.
- Configuration readers.

- Runtime state transitions.

For example, the Automation output contract touches "workflowWalk.ts", run writes, persisted records, output mappings, and Wait/resume handling.

The implementation architecture establishes where the governing decisions are supposed to take effect.

18.3 Conformance architecture

This establishes how we know the implementation satisfies its governing decisions.

It connects decisions to their concrete implementing mechanisms and verification evidence.

For example:

Decision: Outputs produced by a completed module operation remain accessible to downstream nodes.

Implementation: Run-scoped write references and persisted execution state.

Verification: Execute Audit, park at Wait, resume, and confirm the downstream node can still reference the correct Audit records without guessing or repeating the operation.

The conformance relationship matters as much as the documents and the code individually.

Without it, we must repeatedly reconstruct whether a feature really implements the intended behaviour.

With it, Implementation Fidelity can become a systematic check.

---

1. Decision Fit as a constrained research process

The same architecture can govern new recommendations.

When research exposes a failure mode, the process should not immediately ask the founder how the product should behave.

It should first identify the existing decisions applicable to the failure.

Then it should examine established engineering solutions.

A candidate solution must satisfy the existing invariants.

Solutions that contradict them should be rejected, regardless of how common those solutions are elsewhere.

The resulting process is:

"Engineering problem"

↓

"Identify governing decisions"

↓

"Derive required behaviour"

↓

"Find established implementations"

↓

"Evaluate candidates against constraints"

↓

"Select the smallest fitting solution"

↓

"Verify conformance"

This gives external research a very specific role.

It does not independently define the product.

It supplies proven mechanisms for achieving behaviours already established by the product.

19.1 The importance of engineering degrees of freedom

Every product decision fixes some things and leaves others open.

For example:

Fixed requirement| Engineering freedom
Run performs immediate execution in the intended environment| How execution is dispatched and recovered
A node can reference its producing operation's output| How durable output references are represented
Launch arms a workflow| How subscriptions are materialized and monitored

Exhaustion stops spending but preserves completion recognition| How state changes are transacted
Agent rules inherit defaults according to their scope| How effective configuration is resolved efficiently

The research should operate within those remaining degrees of freedom.

That is not a limitation on engineering creativity.

It prevents engineers from spending time resolving decisions that are no longer theirs to make.

---

1. Why the decision model must be relational

The final architectural observation concerned the nature of decisions themselves.

A collection of Markdown documents is primarily organized around files and chronology.

But governing decisions have relationships that cut across files.

An Automation decision may constrain an Agent handoff.

An Agents decision may prohibit a particular Automation validation gate.

A consent decision may override otherwise valid workflow execution.

A later founder correction may supersede language in an earlier specification.

A proposed engineering change may satisfy one decision while violating another.

Therefore, the decision architecture needs to represent these relationships explicitly.

Examples include:

- Governs: this decision governs this module or capability.
- Constrains: this decision limits this interface or operation.
- Depends on: this decision requires another established behaviour.
- Supersedes: this later decision replaces an earlier one.
- Implemented by: these source mechanisms realize the decision.
- Verified by: these checks demonstrate conformance.
- Conflicts with: this proposal violates an existing invariant.

This does not necessarily require a literal graph database.

The key is to make the relationships operationally accessible and maintainable.

The architecture must make it difficult to implement or recommend something while remaining unaware of the decisions that govern it.

---

1. What this discovery means for the recent research initiative

We began the broader initiative because implementations were repeatedly failing to conform to established decisions.

The proposed remedy was to use research into mature production systems to reduce unnecessary invention and improve engineering quality.

That strategy remains useful.

But the conversation revealed a missing prerequisite.

External research cannot guarantee Implementation Fidelity unless its recommendations are evaluated against the original decisions.

And it cannot reliably produce Decision Fit unless it knows which decisions constrain the problem.

We therefore need to distinguish three different kinds of evidence.

Product evidence

What has Tower already decided?

This comes from authoritative founder decisions, specifications, historical corrections, and product methodology.

Implementation evidence

What does Tower's code actually do?

This comes from source inspection, database behaviour, runtime execution, and tests.

Prior-art evidence

How have established systems solved comparable technical problems?

This comes from external source code, libraries, SDKs, documentation, and production tests.

The three sources play different roles.

They should not be treated as competing, interchangeable answers.

A sound engineering recommendation must be compatible with all three, with the product decisions determining required behaviour.

---

1. What is settled, what is derived, and what remains open

It is important not to turn this discussion into an additional layer of unsupported architectural claims.

22.1 Settled or source-confirmed product decisions

The discussion recovered established decisions that include:

- Automations owns event-driven orchestration of real application operations.
- Agents owns campaign, objective, sequencing, attempts, escalation, and content.
- Agents sequences are not governed internally by an Automation Action's selected objective.
- A completed Automation Action must expose outputs attributable to its particular execution.
- The workflow has two distinct mechanisms for handling completed work: the output handle within the same chain and events available to other automations.
- Rule outputs can contain multiple independently meaningful classes.
- Agent rules inherit through defined configuration scopes.
- Exhaustion stops outbound expenditure while preserving recognition of legitimate later completion.
- Objective completion must correspond to the correct objective instance.
- Trigger test data can come from existing event records or versioned specimens.
- Providing test data does not execute the workflow.
- Run executes the workflow immediately.

- Launch arms event-driven execution.
- Run to here is a supported operation.

These decisions provide substantial direction without further founder clarification.

22.2 Conclusions derived from those decisions

Several further requirements follow from the established methodology:

- A valid downstream output reference should not become meaningless solely because execution parks at Wait.
- Infrastructure retries and separately intended business operations require different treatment.
- Graph validation must preserve the behaviours Tower promises, rather than merely matching the current walker's convenience.
- Engineering recommendations must respect the boundary between Automations orchestration and Agents authorship.
- A product recommendation should not be accepted solely because an established external system uses it.
- Historical supersession matters when reconstructing current product law.

These conclusions are grounded in the product model, even where their particular implementation has not yet been decided.

22.3 Technical questions that remain

Examples include:

- How best to persist or reconstruct execution context across Wait.
- How to prevent duplicate external effects after partial failures.
- Whether the current Manual Run environment is sufficiently isolated from unintended production effects.
- How to preserve stable execution definitions while handling intentionally live policy checks.
- How to make graph validation enforce the selected product-supported topology.

- How to maintain accurate decision-to-code and decision-to-test mappings.
- How to verify that an inherited client configuration is actually the one used at execution time.

These are principally engineering and conformance problems.

They do not automatically require new product decisions.

22.4 Genuine product choices still possible

Some choices may genuinely remain unmade—for example, a category-specific policy about whether an already-running execution should adopt a newly published non-safety configuration.

But the proper process is to establish that the choice cannot be derived from existing decisions before elevating it.

The fact that a recent research document labels something unresolved is insufficient evidence of a genuine product silence.

---

1. The final synthesis

The conversation began with workflow-graph research identifying bugs, ambiguous execution behaviour, and possible engineering improvements.

We initially examined these as ordinary implementation and architecture questions.

The user challenged that framing by asking whether the deeper methodology had already resolved them.

Investigation revealed that many had indeed been decided, sometimes in remarkable technical detail.

The Test/Run/Launch investigation made the pattern unmistakable.

What looked like a product ambiguity had already been discussed, revised, documented, and implemented.

The correct question was not what Run should mean.

It was whether the implementation faithfully represented what Run had already been defined to mean.

That gave the first concept its name.

Implementation Fidelity.

The same experience exposed a second problem.

When an engineer encounters a new failure mode, they may propose a locally reasonable solution that contradicts the surrounding product architecture.

The challenge is not merely finding a technically effective answer.

It is finding one that belongs within the existing system of decisions.

That gave the second concept its name.

Decision Fit.

Together they describe the two directions of the same architectural problem.

One asks whether an implementation conforms to its governing decisions.

The other asks whether a proposed decision conforms to the governing system.

Both require the same underlying capability:

Authoritative decisions must be structurally connected to the implementations, proposals, and verification mechanisms they govern.

---

1. The architecture we arrived at

The proposed structure can be expressed as:

```
              DECISION ARCHITECTURE
              ---------------------
              Authoritative decisions
              Rationale
              Scope and ownership
              Invariants
              Dependencies
              Supersession
              Engineering freedom
                      |
         +------------+------------+
         |                         |
         v                         v
IMPLEMENTATION FIDELITY       DECISION FIT
-----------------------       ------------
Established decision          New problem
         |                         |
         v                         v
Implementing mechanisms       Applicable decisions
```

```
         |                          |
         v                          v
Conformance verification       Prior-art candidates
         |                          |
         v                          v
Does implementation            Does proposed solution
fulfill the decision?          fit the system?
         |                          |
         +------------+-------------+
                      |
                      v
              IMPLEMENTATION
              ARCHITECTURE
              --------------
              Source code
              APIs
              Data
              Runtime behaviour
              External systems
                      |
                      v
              CONFORMANCE EVIDENCE
              --------------------
              Tests
              Execution traces
              Verified invariants
              Failure-mode checks
```

The architecture has not yet been implemented.

This is the conceptual result of the discussion.

The important discovery is the shared underlying decision structure, not the particular arrangement of boxes or any specific storage technology.

---

1. The deeper principle

The fundamental problem is not that engineers lack enough context.

It is that established decisions can fail to exert authority over subsequent work.

A decision may exist in a founder log and still be absent from a code review.

It may exist in a Product Manager packet and still be ignored by a runtime implementation.

It may exist in an earlier correction and still be contradicted by a later research recommendation.

These failures are not repaired simply by writing the same decision more extensively.

They are repaired by making the governing relationships identifiable, actionable, and verifiable.

That is why Implementation Fidelity and Decision Fit belong together.

Implementation Fidelity asks whether reality conforms to intent.

Decision Fit asks whether a proposed change conforms to intent.

Both depend on a system that knows what the intent actually governs.

And the purpose of that architecture is ultimately practical:

Reduce the number of decisions that must be made again, reduce the number of implementations that drift from settled decisions, and preserve engineering freedom where the product has deliberately left it open.

---

1. Research and source trail

The following resources were important to the discussion.

Resource| Contribution
"PR #131 — Workflow Graph Prior Art" (https://github.com/bywale-com/omcoda_tower/pull/131)| Established graph execution discrepancies and external implementation references
"docs/register/PHASE-TWO-LOG.md"| Preserved original module decisions and later founder corrections
"docs/register/AGENTS-FACE.md"| Established Agents authorship, campaign hierarchy, and overrides
"docs/register/AUTOMATIONS-FACE.md"| Established orchestration, rule outputs, events, and node-output semantics
"docs/register/PRIMITIVES.md"| Defined what Automations may hear, perform, and pass downstream
"docs/register/SYSTEMS-AUTOMATIONS-NODE-OUTPUT.md"| Defined the identity and availability of module-produced writes
"docs/register/SYSTEMS-CHANGE-PACKET-022.md"| Established objective completion as a correctly attributed Local event
"docs/register/SYSTEMS-CHANGE-PACKET-023.md"| Established exhaustion as stopping pursuit, not stopping completion recognition

"PR #99 — Event Test Contract" (https://github.com/bywale-com/omcoda_tower/pull/99)| Implemented versioned event write-sets and test specimens
"PR #105 — Test/Run/Launch Split" (https://github.com/bywale-com/omcoda_tower/pull/105)| Implemented the revised separation of test input, immediate execution, and activation
"PR #96 — Original PM Packets" (https://github.com/bywale-com/omcoda_tower/pull/96)| Preserved the original Test and Run decisions
"supabase/migrations/017_event_test_contract.sql"| Versioned event-contract persistence
"server/wire-service/eventTest.ts"| Current event test-data reader
"server/wire-service/workflowWalk.ts"| Actual workflow execution semantics
"server/wire-service/machine.ts"| Manual Run, Save, and Launch server endpoints
"server/wire-service/cascade.ts"| Enrollment and attempt creation
"server/wire-service/sendMode.ts"| Outbound dry-run and live-send gating

---

1. Final statement of the discovery

The original initiative sought better engineering by studying how proven systems solve the problems Tower encounters.

That remains worthwhile.

But the discussion exposed the missing architectural principle:

Before researching how to implement something, identify what has already been decided about it. Before proposing a solution, establish how it fits those decisions. After implementing it, establish whether the implementation is faithful to them.

This produces two permanent concerns:

Implementation Fidelity

Did we build what we decided?

Decision Fit

Does what we're about to decide belong in what we've already built and decided?

Neither should depend on repeatedly asking the founder to reconstruct the reasoning from memory.

Both should emerge from an architecture capable of preserving decision authority across the entire engineering lifecycle.

The problem has moved from insufficient context to the structural governance of decisions.

That is the central discovery of this discussion.